

Heurísticas de Encaixe para o Coupled-Task Scheduling Problem: Propostas de restrições adicionais para minimização do makespan

Pedro Canellas Innecco

Kelwyn Filipi Oliveira Reis de Souza

André Renato Villela da Silva

Resumo

O Coupled-Task Scheduling Problem (CTSP) consiste em escalonar N tarefas independentes em uma única máquina, cada uma composta por duas partes a_i e b_i , separadas por um intervalo fixo l_i , com o objetivo de minimizar o makespan. Trabalhos recentes comparam modelagens de Programação Inteira Mista (PIM) e Programação por Restrições (CP) para o problema, evidenciando a eficiência de modelos em CP na obtenção de soluções exatas — entre eles, a formulação proposta por Silva (2025), adotada neste trabalho como base de comparação. Neste Trabalho de Conclusão de Curso, são propostas e avaliadas heurísticas construtivas baseadas em estratégias de encaixe (nesting) de tarefas dentro dos intervalos ociosos do problema. A pesquisa partiu de uma heurística recursiva inicial, que seleciona a tarefa com maior intervalo da instância como âncora e encaixa sucessivamente a maior tarefa disponível que caiba no espaço remanescente, e evoluiu para uma versão aprimorada dessa lógica, na qual o algoritmo testa sistematicamente a remoção e a recolocação de cada tarefa do bloco encaixado, utilizando a melhor configuração encontrada como solução inicial para o solver. As soluções obtidas por essas heurísticas são comparadas com as soluções da formulação em CP de Silva (2025), utilizando instâncias de diferentes tamanhos. Os resultados indicam que as heurísticas de encaixe propostas produzem makespans competitivos com a modelagem base, superando-a em parte significativa das instâncias testadas, e se mostram promissoras como geradoras de boas soluções para o CTSP.

Palavras-chave: Programação por Restrições; CTSP; Heurísticas de Encaixe; Aninhamento de Tarefas;

Abstract

The Coupled-Task Scheduling Problem (CTSP) consists of scheduling N independent tasks on a single machine, each task composed of two parts a_i and b_i separated by a fixed interval l_i , with the objective of minimizing the makespan. Recent studies compare Mixed-Integer Programming (MIP) and Constraint Programming (CP) formulations for the problem, highlighting the effectiveness of CP models in obtaining exact solutions — among them, the formulation proposed by Silva (2025), adopted in this work as a benchmark for comparison. In this undergraduate thesis, constructive heuristics based on nesting tasks inside the idle intervals of the problem are proposed and evaluated. The research started from an initial recursive heuristic, which selects the task with the largest interval in the instance as an anchor and successively nests the largest available task that fits into the remaining space, and evolved into an improved version of this logic, in which the algorithm systematically tests the

removal and reinsertion of each task in the nested block, using the best solution found as the initial solution for the solver. The solutions obtained by these heuristics are compared with the solutions from Silva's (2025) CP formulation, using instances of different sizes. The results indicate that the proposed nesting heuristics produce competitive makespans when compared to the base model, outperforming it in a significant portion of the tested instances, and show promise as generators of good solutions for the CTSP.

Keywords: Constraint Programming; CTSP; Embedding Heuristics; Task Nesting;

1 INTRODUÇÃO

O escalonamento de tarefas é um problema clássico da computação que busca responder a uma pergunta simples: dado um conjunto de tarefas que precisam ser executadas, qual é a melhor ordem para realizá-los? Dentro desse campo, existe uma classe particular de problemas em que cada tarefa não é uma parte única, mas composta por duas partes (a_i e b_i) que devem ser executadas com um intervalo de tempo exato (l_i) entre elas. Esse tipo de problema é conhecido como Problema de Escalonamento de Tarefas Acopladas, ou, internacionalmente, Coupled-Task Scheduling Problem (CTSP).

O CTSP foi introduzido por Shapiro (1980) para modelar sistemas de radar de pulso, nos quais um sinal é emitido, aguarda-se um intervalo fixo até o eco retornar, e só então o sinal é processado. Nesse modelo, cada tarefa é composta por uma parte inicial e uma parte final, ambas com tamanhos fixos e determinados, que devem ser executadas necessariamente nessa ordem e separadas por um intervalo de tempo igualmente fixo. O desafio central do CTSP é organizar essas tarefas de forma a minimizar o tempo total necessário para concluir todo o conjunto de tarefas, medida conhecida como makespan.

A Figura 1 mostra um possível escalonamento para três tarefas, onde os tempos da tarefa 1 são $a_1 = 9$, $l_1 = 49$ e $b_1 = 11$. A tarefa 2 é composta por $a_2 = 8$, $l_2 = 13$ e $b_2 = 12$. A tarefa 3 possui os tempos $a_3 = 10$, $l_3 = 20$ e $b_3 = 10$. Como o intervalo da tarefa 1 é significativo, utiliza-se dele para executar outras tarefas e tentar minimizar o makespan que, nesse exemplo, é igual a 75 unidades de tempo.

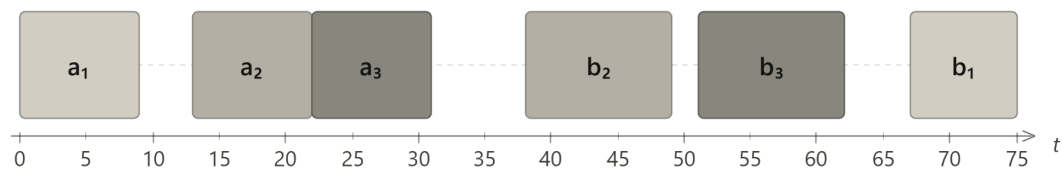


Figura 1: Exemplo de escalonamento de tarefas.

Um ponto fundamental do problema é que as tarefas são tratadas como independentes entre si: não existe relação de precedência ou dependência entre as diferentes tarefas, ou seja, a ordem de execução de uma tarefa não depende da conclusão de outra. Essa independência, contudo, não deve ser confundida com liberdade total de ação durante os intervalos de espera. A máquina realmente não precisa ficar ociosa enquanto uma tarefa aguarda seu intervalo — ela pode aproveitar esse tempo para executar partes de outras tarefas. Essa flexibilidade, no entanto, está sujeita a uma restrição rígida, chamada de não preempção: uma vez iniciada a execução de qualquer parte de uma tarefa, ela deve ser concluída integralmente, sem interrupções ou sobreposições com outras tarefas.

Essa combinação de exigências — intervalos exatos, independência entre tarefas e impossibilidade de interrupção — torna o CTSP um problema computacionalmente difícil. Sua complexidade é classificada como NP-Difícil (Shapiro, 1980), o que, em termos práticos, significa que, à medida que o número de tarefas aumenta, encontrar a solução ótima exige um esforço computacional que cresce de forma extremamente rápida, tornando-se inviável testar todas as possibilidades em tempo hábil.

Apesar dessa dificuldade, o CTSP não é apenas um problema teórico: ele descreve situações reais em diversas áreas. Na área militar, por exemplo, o modelo é usado no controle de sensores para guiar torpedos submarinos (Simonin et al., 2011). Na indústria química, ele se aplica a processos de manufatura nos quais tempos de espera exatos são necessários para que reações ocorram corretamente entre uma etapa e outra (Ageev e Kononov, 2007). Mais recentemente, o problema também tem sido empregado para otimizar o agendamento de consultas de quimioterapia, situação em que o intervalo entre as doses precisa seguir rigorosamente um protocolo médico (Condotta e Shakhlevich, 2014).

Diante dessa complexidade computacional, a literatura sobre o tema tem evoluído de métodos exatos — que garantem a solução ótima, mas se tornam impraticáveis para problemas grandes — para heurísticas híbridas, que combinam diferentes estratégias para encontrar soluções de boa qualidade em tempo razoável, mesmo sem garantia de otimalidade. É nesse contexto que se insere o presente trabalho, que propõe aprimorar uma formulação matemática existente para o CTSP por meio de “Heurísticas de Encaixe”.

Este trabalho está organizado em cinco seções, incluindo esta introdução. A Seção 2 apresenta a revisão da literatura, na qual são discutidas as principais formulações e aplicações do CTSP exploradas ao longo dos anos, com foco especial na modelagem de Programação por Restrições proposta por Silva (2025), adotada como base de comparação para os

resultados desta pesquisa. A Seção 3 descreve a metodologia adotada, detalhando o processo experimental de construção e refinamento das heurísticas de encaixe desenvolvidas neste trabalho, desde a concepção inicial até sua evolução. A Seção 4 apresenta os resultados obtidos, comparando o desempenho das heurísticas propostas com o da formulação base em instâncias de diferentes tamanhos. Por fim, a Seção 5 traz as conclusões do trabalho, sintetizando as principais contribuições alcançadas e apontando possíveis direções para pesquisas futuras.

2 REVISÃO DA LITERATURA

A estrutura fundamental do CTSP impõe uma restrição rígida de tempo: a segunda parte de uma tarefa só pode ser iniciada exatamente no instante em que o intervalo definido entre suas duas partes se encerra — nem antes, nem depois. Essa exigência, combinada com a impossibilidade de interromper tarefas ou sobrepor execuções na máquina, transforma o escalonamento em um exercício de coordenação precisa entre os tempos de início e término de cada parte de cada tarefa.

Para aproveitar o tempo ocioso gerado por esses intervalos fixos, Shapiro (1980) propôs duas técnicas principais de organização: a intercalação (interleaving) e o aninhamento (nesting). Na intercalação, as tarefas são dispostas de modo que suas partes finais sigam a mesma ordem cronológica de suas partes iniciais — ou seja, se a tarefa A começa antes da tarefa B, a parte final de A também é executada antes da parte final de B. Já no aninhamento, uma tarefa inteira (incluindo seu próprio intervalo) é encaixada completamente dentro do tempo ocioso de uma tarefa maior, invertendo essa ordem: a tarefa que começou depois termina antes. Essas duas técnicas formam a base sobre a qual algoritmos mais sofisticados de escalonamento são construídos, já que determinam como o tempo ocioso de cada tarefa pode ser reaproveitado sem violar as regras de não preempção.

Uma abordagem eficiente para tratar instâncias de grande escala, combinando essas ideias de forma sistemática, é a Heurística de Busca Binária (Khatami e Salehipour, 2021). Essa metodologia parte de dois limites para o makespan: um limite inferior, correspondente ao tempo mínimo teoricamente possível, e um limite superior, obtido a partir de um escalonamento simples e já viável. A partir daí, o algoritmo testa repetidamente se existe uma solução viável para um tempo "alvo" situado entre esses dois limites. Quando uma solução é encontrada, o alvo é reduzido, na tentativa de aproximar-se ainda mais do tempo mínimo;

quando não é encontrada, o alvo é aumentado. Esse processo de tentativa e ajuste estreita progressivamente o intervalo entre os limites até convergir para a melhor solução possível.

Para fortalecer essa busca, são empregadas técnicas de Programação Inteira — uma abordagem matemática que busca a melhor solução para um problema dentro de um conjunto de restrições, exigindo que as variáveis de decisão assumam exclusivamente valores inteiros em sua solução final — com o objetivo de calcular quanto de um intervalo ocioso pode efetivamente ser preenchido por outras tarefas.

A formulação utilizada como base neste trabalho é a modelagem de Programação por Restrições (Constraint Programming — CP) proposta por Silva (2025), para resolver o CTSP com intervalos exatos em uma única máquina. Diferentemente das formulações tradicionais de Programação Inteira Mista, que representam o tempo por meio de variáveis numéricas, essa abordagem utiliza variáveis do tipo intervalo — um recurso disponível em *solvers* de Programação por Restrições, como o CP Optimizer, que representam diretamente um período de tempo com ponto de início, ponto de fim e duração fixa. Cada tarefa i é decomposta em dois desses intervalos, correspondentes à sua parte inicial (a_i) e à sua parte final (b_i), sobre os quais incidem as restrições do problema. A primeira é a restrição de acoplamento exato, que impõe que o início da parte final ocorra precisamente no instante em que termina a parte inicial somado ao tempo de latência fixo (l_i) entre as duas, garantindo que o intervalo entre as partes de cada tarefa seja respeitado. Também é considerada a condição de não sobreposição, que garante que, havendo apenas uma máquina disponível, nenhum intervalo de tempo seja ocupado por mais de uma parte simultaneamente. Há ainda a exigência de execução obrigatória, na qual todas as tarefas do problema devem necessariamente ser executadas.

A função objetivo do modelo é a minimização do makespan, definido como o maior tempo de encerramento entre todas as partes b_i do problema. A formulação matemática completa proposta por Silva (2025) é apresentada a seguir:

minimize

$$\max(\text{end}(b_i)) \quad (1)$$

s.t.

$$\text{start}(b_i) = \text{end}(a_i) + l_i \quad i \in N \quad (2)$$

$$\text{presence}(a_i, b_i) = \text{true} \quad i \in N \quad (3)$$

$$\text{noOverlap}(i, i') \quad (i, i') \in N, i < i' \quad (4)$$

$$\text{length}(a_i) = A_i \quad i \in N \quad (5)$$

$$\text{length}(b_i) = B_i \quad i \in N \quad (6)$$

A função-objetivo (1) representa a minimização do makespan, obtido a partir do maior tempo de encerramento entre todas as partes b_i do problema. As restrições (2) garantem o acoplamento exato entre as duas partes de cada tarefa, exigindo que a parte final b_i comece exatamente l_i unidades de tempo após o término da parte inicial a_i . As restrições (3) asseguram que todas as tarefas do problema sejam efetivamente executadas. As restrições (4) impõem a condição de não sobreposição entre quaisquer duas tarefas distintas, garantindo que a máquina nunca execute mais de uma parte simultaneamente — vale notar que a restrição de não preempção não precisa ser modelada explicitamente, já que é garantida automaticamente pela própria natureza das variáveis do tipo intervalo utilizadas nessa formulação. Por fim, as restrições (5) e (6) fixam as durações das partes inicial e final de cada tarefa, respectivamente. Conforme demonstrado por Silva (2025), essa modelagem baseada em Programação por Restrições se mostrou superior às formulações tradicionais de Programação Inteira Mista, encontrando, em geral, limites melhores para o makespan em tempos computacionais menores.

3 METODOLOGIA

A metodologia deste trabalho fundamenta-se na busca por um escalonamento eficiente que aproveite melhor os intervalos ociosos do CTSP, reduzindo ao máximo o tempo da máquina desperdiçado entre as duas partes de cada tarefa. Partindo da formulação de Silva (2025) como ponto de comparação, esta pesquisa busca superar seus resultados por meio do desenvolvimento de heurísticas de encaixe, que serão atribuídas como restrições para o solver, e que serão melhor detalhadas a seguir. A ideia central que orienta todas as abordagens aqui apresentadas tem origem direta na técnica de aninhamento (nesting), descrita anteriormente: se partes de outras tarefas podem ser encaixadas dentro do intervalo ocioso de uma tarefa maior, então é possível buscar, de forma sistemática, a configuração de encaixe que minimize o tempo ocioso restante — e, conseqüentemente, o makespan da instância. Cada uma dessas heurísticas resulta em um subconjunto de tarefas que deve ser escalonado em uma ordem específica; esse escalonamento é então imposto ao solver por meio de restrições adicionais, que forçam o início das tarefas conforme o posicionamento previamente computado pela heurística. Para alcançar esse objetivo, a pesquisa evoluiu por meio de um processo experimental de construção e refinamento sucessivo de heurísticas, descrito nas subseções a seguir.

3.1. Heurística Inicial e Evolução dos Blocos Âncora

A abordagem inicial, identificada como V1, baseou-se na lógica de aninhamentos sucessivos aplicados de forma recursiva: a tarefa com o maior intervalo entre suas partes, em toda a instância, era considerada o único bloco-âncora utilizado pela heurística. Por questões de flexibilidade na execução do modelo, optou-se por trabalhar com apenas um bloco-âncora por instância, em vez de permitir que outras tarefas de grande intervalo também assumissem esse papel simultaneamente. A partir dessa âncora, buscava-se, entre as tarefas candidatas, aquela cuja duração total — entendida como a soma de suas duas partes mais o próprio intervalo entre elas — fosse a maior possível, desde que ainda igual ou menor ao intervalo disponível na âncora. Uma vez encontrada e encaixada essa tarefa, o mesmo raciocínio era aplicado novamente de forma recursiva: o intervalo ocioso remanescente, agora menor, passava a ser tratado como referência para a busca da próxima maior tarefa que nele coubesse, e assim sucessivamente, até que não houvesse mais tarefas candidatas compatíveis com o espaço restante. A sua formulação consiste em:

$$start(a_{i+1}) \geq end(a_i) \quad i \in 1.. \|L\| - 1 \quad (7)$$

$$end(b_{i+1}) \leq start(b_i) \quad i \in 1.. \|L\| - 1 \quad (8)$$

As restrições (7) e (8) formalizam a estrutura da V1 para uma lista ordenada L . A restrição (7) impõe que as operações iniciais a_i sejam executadas na ordem direta da lista, isto é, cada a_{i+1} só pode iniciar após o término de a_i . Já a restrição (8) impõe que as operações finais b_i sejam executadas na ordem inversa, ou seja, cada b_{i+1} deve terminar antes do início de b_i . Em conjunto, essas restrições garantem que os tempos de espera das tarefas fiquem aninhados uns dentro dos outros, de forma que a tarefa 1 englobe integralmente o intervalo de espera da tarefa 2, que por sua vez englobe o da tarefa seguinte, e assim sucessivamente até a última tarefa da lista. Essa propriedade de aninhamento é o que permite aproveitar os gaps já formados na sequência para inserir novas tarefas sem violar a precedência entre as operações a e b de cada tarefa.

A Figura 2 ilustra um possível escalonamento para três tarefas segundo a modelagem V1. No eixo do tempo, a_1 , ocupa o intervalo $[0, 10]$, a_2 $[10, 20]$ e a_3 $[20, 30]$, executadas em sequência direta. Em seguida, as operações finais são executadas na ordem inversa: b_3 ocupa $[30, 40]$, b_2 $[40, 55]$ e b_1 $[55, 65]$. Note que b_3 inicia logo após a_3 , sem tempo de espera, enquanto b_2 e b_1 apresentam intervalos de espera maiores (20 e 45 unidades de tempo, respectivamente), gerando o padrão de encaixamento visível na figura, em que o intervalo de b_3 está contido no de b_2 , que por sua vez está contido no de b_1 .

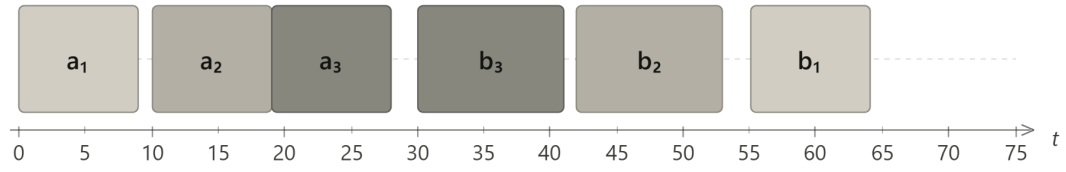


Figura 2: Exemplificação da modelagem V1.

Com base nessa heurística inicial, foram pensadas três variantes. A variante chamada de V1.1 não restringia a busca à maior tarefa individual que coubesse no intervalo, mas passou a considerar também a possibilidade de agrupar múltiplas tarefas — ou até mesmo manter uma única tarefa, quando essa fosse a melhor opção — selecionando, entre as combinações possíveis, aquela que resultasse no menor tempo ocioso remanescente, ou seja, a que minimizasse a diferença entre o intervalo disponível na âncora e a soma das durações totais das tarefas agrupadas nela. Formalmente, a V1.1 pode ser expressa por:

$$start(a_{l_i}) \geq end(a_{âncora(1)}) \quad i \in 2.. \|L\| \quad (9)$$

$$end(b_{l_i}) \leq start(b_{âncora(1)}) \quad i \in 2.. \|L\| \quad (10)$$

As restrições (9) e (10) definem como a lista L é encaixada dentro do intervalo de espera da tarefa âncora. A restrição (9) exige que a operação inicial de cada tarefa l_i da lista ($i = 2, \dots, \|L\|$) comece somente após o término de $a_{âncora(1)}$, garantindo que toda a lista seja posicionada dentro do gap aberto pela âncora. Já a restrição (10) exige que a operação final de cada tarefa l_i termine antes do início de $b_{âncora(1)}$, fechando o encaixamento pelo outro lado. Juntas, essas restrições garantem que toda a lista L — com sua própria estrutura interna de aninhamento, dada pelas equações (7) e (8) — fique inteiramente contida dentro do intervalo de espera $[end(a_{âncora(1)}), start(b_{âncora(1)})]$ da tarefa âncora, preservando a precedência entre as operações a e b tanto da âncora quanto das tarefas inseridas.

A variante V1.2 testou uma divisão igualitária do tempo de processamento entre essa lógica original da V1 e da V1.1 recém-descrita, numa tentativa de equilibrar os dois critérios de encaixe durante a execução do algoritmo. A variante V1.3, por fim, restringiu o universo de busca dessa mesma lógica de múltiplas tarefas (V1.1) a apenas uma faixa específica das tarefas disponíveis: aquelas situadas entre 0% e 20% dos maiores intervalos do conjunto completo de tarefas.

3.2. Modelos de Correntes

Após os testes realizados com a V1 e suas variantes, a pesquisa derivou para uma abordagem distinta, batizada de V2, baseada na criação de elos em corrente entre tarefas. A

ideia central consistia em ligar duas tarefas de modo que o término da primeira parte de uma fosse acoplado exatamente ao início da segunda parte de outra, formando um fluxo contínuo de execução, sem espaços ociosos entre as duas, conforme exemplificado na Figura 3. Nela, a_1 , ocupa o intervalo $[0, 5]$ e b_2 ocupa o intervalo $[55, 60]$, mantendo seus próprios tempos de espera isolados. Já a_2 , no intervalo $[20, 25]$, e b_1 , no intervalo $[25, 30]$, formam o elo da corrente: o término de a_2 coincide exatamente com o início de b_1 , eliminando o tempo ocioso entre essas duas operações e caracterizando o encadeamento que dá nome à modelagem V2.

Formalmente, essa relação é expressa por:

$$end(a_{l_2}) = start(b_{l_1}) \quad l_1, l_2 \in (l_1, l_2) \quad (11)$$

A equação (11) define o elo da corrente: o término da operação inicial da tarefa l_2 deve coincidir exatamente com o início da operação final da tarefa l_1 . É essa igualdade que garante o fluxo contínuo de execução entre as duas tarefas acopladas, sem espaço ocioso entre elas, conforme ilustrado na Figura 3.

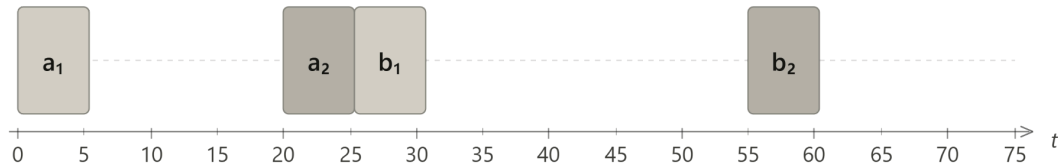


Figura 3: Exemplificação da modelagem V2.

Essa técnica também foi testada em diferentes variações. A variante V2.1 investigou diferentes percentuais de tarefas da instância que seriam submetidas a esse tipo de acoplamento, buscando identificar a proporção mais vantajosa. A variante V2.2, chamada de aninhamento triplete, tentou ir além do posicionamento inicial entre duas tarefas, introduzindo uma terceira tarefa, cujo intervalo seria utilizado para executar as duas partes já acopladas das tarefas anteriores — combinando, assim, a lógica de corrente com a lógica de aninhamento original da V1, refletida na Figura 4. Nela, a_1 ocupa o intervalo $[0, 5]$ e b_2 o intervalo $[55, 60]$, formando o par externo nos moldes da V1. Dentro desse espaço, a_3 ocupa $[17, 23]$ e b_3 ocupa $[38, 44]$, envolvendo por sua vez o elo formado por a_2 $[26, 31]$ e b_1 $[32, 36]$, cujo término de a_2 coincide exatamente com o início de b_1 — reproduzindo a lógica de corrente da V2 agora aninhada dentro de um par adicional de tarefas. Essa estrutura é formalizada pelas seguintes restrições:

$$end(a_{l_1}) \leq start(a_{l_2}) \leq start(a_{l_3}) \quad l_1, l_2, l_3 \in (l_1, l_2, l_3) \quad (12)$$

$$end(b_{l_1}) \leq start(b_{l_2}) \leq start(b_{l_3}) \quad l_2, l_3 \in (l_1, l_2, l_3) \quad (13)$$

$$end(a_{l_3}) = start(b_{l_1}) \quad l_1, l_2, l_3 \in (l_1, l_2, l_3) \quad (14)$$

Com a introdução de uma terceira tarefa, a modelagem passa a exigir duas cadeias de precedência, uma para cada tipo de operação. A equação (12) estabelece que as operações iniciais devem obedecer à ordem $l_1 \rightarrow l_2 \rightarrow l_3$, isto é, a_1 termina antes de a_2 começar, que por sua vez começa antes de a_3 . A equação (13) impõe a mesma lógica de ordenação sequencial para as operações finais ($b_1 \rightarrow b_2 \rightarrow b_3$). Já a equação (14) reproduz o elo da corrente original da V2, porém deslocado para o par mais externo do encaixamento: agora é o término de a_3 que deve coincidir exatamente com o início de b_1 , unindo a terceira tarefa — mais recentemente inserida — à primeira, mais externa, enquanto l_2 fica aninhada entre as duas.

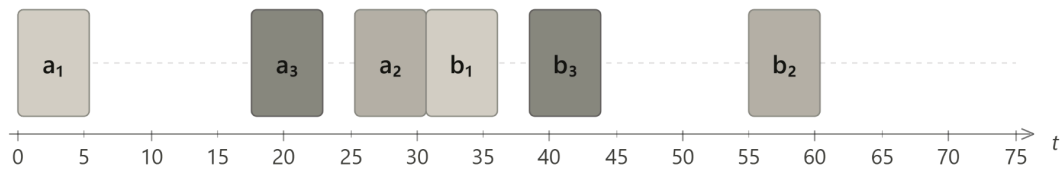


Figura 4: Exemplificação da V2.2.

Por fim, a variante V2.3 relaxou a exigência de que as partes das duas primeiras tarefas estivessem perfeitamente coladas uma à outra, permitindo a existência de pequenas lacunas entre elas, na tentativa de avaliar se essa flexibilidade adicional resultaria em ganhos de eficiência, como no exemplo da Figura 5. Nela, a_1 mantém-se em $[0, 5]$ e b_2 em $[55, 60]$, enquanto a_3 $[17, 22]$ e a_2 $[23, 27]$ permanecem adjacentes. A diferença em relação à V2.2 aparece nos elos internos: em vez do encaixe perfeito observado anteriormente, agora há uma lacuna de 5 unidades entre o término de a_2 e o início de b_1 $[33, 38]$, assim como uma pequena folga entre o término de b_1 e o início de b_3 $[39, 44]$ — ilustrando a flexibilidade introduzida por essa variante.

A V2.3 mantém as restrições de ordenação sequencial entre as operações iniciais (12) e finais (13), descritas na variante anterior, porém remove a exigência de igualdade exata da equação (14). Isso significa que o encaixe entre a_3 e b_1 deixa de ser obrigatoriamente perfeito, passando a admitir folgas entre o término de uma operação e o início da outra — flexibilização que caracteriza justamente a proposta dessa variante, testada para avaliar se a folga adicional traria ganhos de eficiência ao escalonamento.

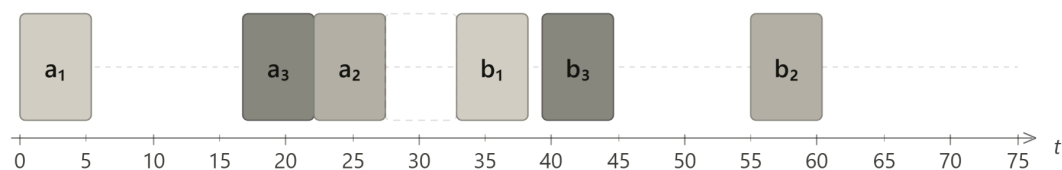


Figura 5: Exemplificação da V2.3.

3.3. Heurística Final

Diante dos resultados obtidos com as metodologias anteriores, a pesquisa retomou e aprofundou a lógica original dos Blocos Âncora, testando a V1 com a remoção da menor tarefa aninhada (Figura 6). Nela, a_1 ocupa o intervalo $[0, 9]$ e a_2 o intervalo $[10, 19]$, mantendo a mesma disposição das operações iniciais observada na modelagem original. O intervalo $[19, 42]$ permanece ocioso, correspondendo ao espaço que seria ocupado pela terceira tarefa — a mais internamente aninhada — caso ela não tivesse sido removida. Por fim, b_2 ocupa o intervalo $[42, 53]$ e b_1 o intervalo $[55, 64]$, fechando o encaixamento na ordem inversa das operações iniciais.

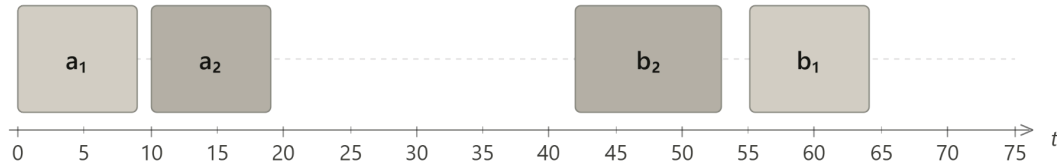


Figura 6: Exemplificação da V1, removendo a menor tarefa aninhada.

A heurística em questão (V1.4) amplia essa ideia realizando uma remoção sistemática de cada tarefa do bloco, uma por vez. Mais especificamente, o algoritmo divide o processo em $n + 1$ etapas, sendo n o tamanho do bloco de tarefas candidatas ao encaixe. Em cada uma das n primeiras etapas, a modelagem é executada com uma das tarefas do bloco temporariamente removida, sendo posteriormente recolocada antes de se passar à etapa seguinte. Na etapa final ($n+1$), nenhuma restrição adicional é imposta ao modelo, e a melhor solução encontrada até esse momento, dentre as etapas anteriores, é utilizada como solução inicial para o solver.

4 RESULTADOS

Os testes computacionais foram realizados para avaliar a eficácia das metodologias propostas na minimização do makespan. Para isso, foi desenvolvido um script em Python responsável por identificar o bloco de tarefas a ser utilizado em conjunto com a modelagem matemática a cada execução. Esse script aciona o solver CP Optimizer 22.1.1.0, executado em uma máquina com processador Intel i7-10700, 16GB de memória RAM e sistema operacional Ubuntu 22.04 LTS. Para garantir a consistência dos experimentos, utilizou-se um limite de tempo de execução de 600 segundos por tarefa. Foram testadas, no total, 150 instâncias para cada método, divididas em grupos de 30 instâncias para cada um dos seguintes tamanhos: 20, 25, 40, 50 e 100 tarefas. A avaliação consistiu em comparar a execução da modelagem original, proposta por Silva (2025), com versões dessa mesma modelagem acrescidas de

restrições adicionais, responsáveis por impor o escalonamento de determinadas tarefas conforme as heurísticas de encaixe apresentadas anteriormente. Para cada execução, foram analisados dois critérios principais: o valor da melhor solução encontrada para o makespan e o tempo necessário para que o solver chegasse a essa solução.

4.1. Desempenho da Heurística Inicial (V1) e Suas Derivações

A primeira fase de testes focou no refinamento sucessivo da heurística dos Blocos Âncora, por meio da lógica de aninhamento recursivo. Observou-se que essa abordagem tende a aproveitar bem o intervalo de uma única tarefa-âncora, mas apresenta dificuldade em se adaptar a instâncias com características distintas, já que toda a lógica de encaixe depende da tarefa de maior intervalo da instância. Ainda assim, a heurística V1 superou a modelagem base em 61 das instâncias testadas (sendo 5 dessas vitórias por tempo de execução). A Figura 7, apresentada a seguir, detalha esses resultados por tamanho de instância, mostrando a quantidade de vitórias e derrotas obtidas pela V1 em cada um dos grupos testados.

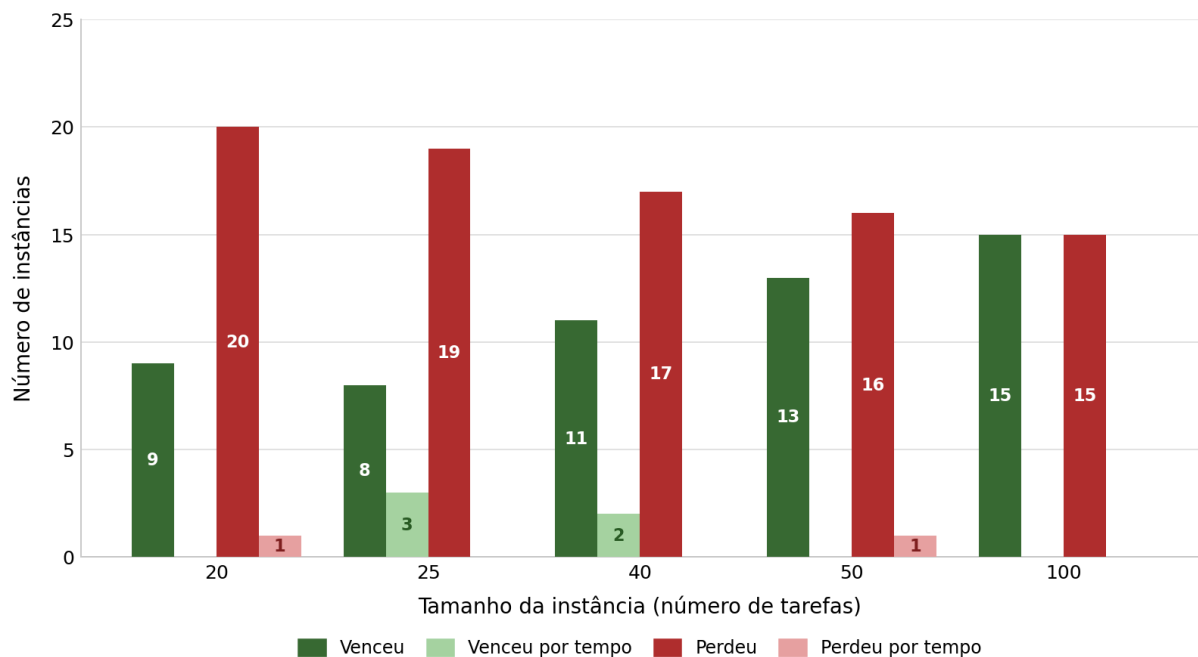


Figura 7: Resultados do modelo V1 inicial por tamanho.

Na variante V1.1, ao tentar encaixar múltiplas tarefas dentro do intervalo da âncora, os resultados se mostraram abaixo da modelagem base, e ligeiramente abaixo da V1, como mostra a Figura 8.

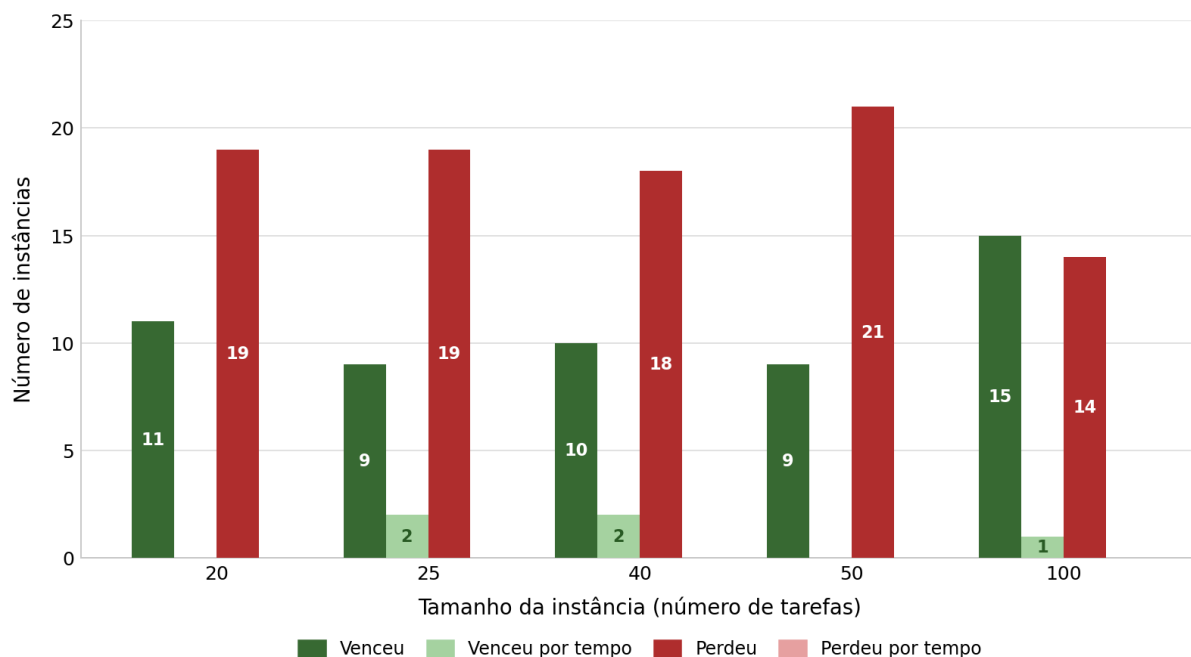


Figura 8: Resultados da V1.1 por tamanho.

Com isso, foi testado dividir o esforço computacional entre a lógica V1 e o modelo de múltiplas tarefas (V1.2), o que não indicou uma melhora significativa, demonstrando que a simples repartição do tempo de processamento não compensa a falta de uma estratégia de posicionamento mais refinada, conforme mostra a Figura 9.

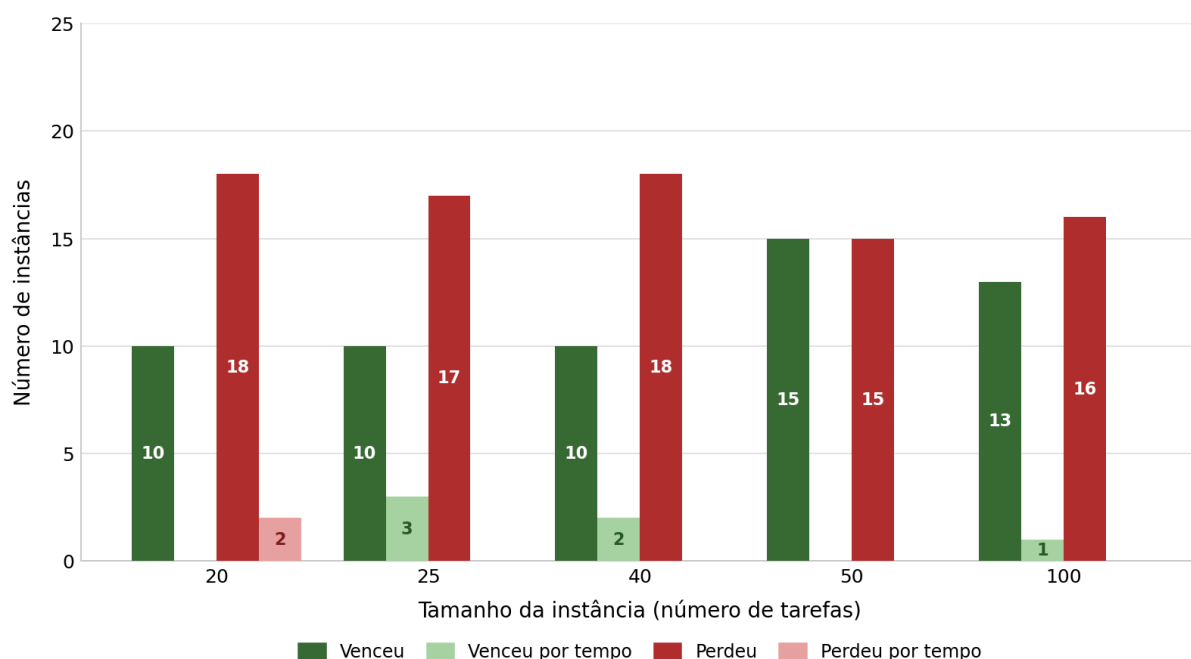


Figura 9: Resultados da execução da V1.2 por tamanho.

Com isso, foi testada a variante V1.3, na qual o algoritmo passou a considerar apenas 20% das tarefas com os maiores intervalos da instância. Os resultados, no entanto, mostraram um desempenho inferior ao das demais variações, conforme mostra a Figura 10.

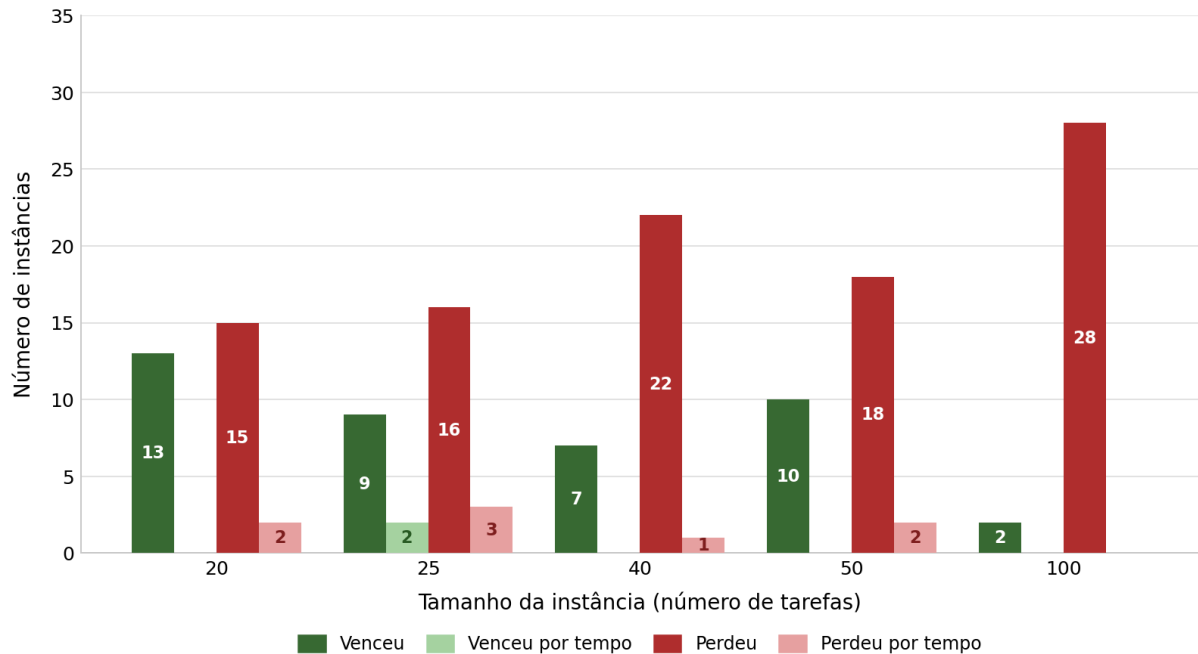


Figura 10: Resultados da execução da V1.3 por tamanho forçando os encaixes até 20% das tarefas com maiores intervalos.

4.2. Desempenho da Heurística V2

A exploração dos modelos V2, baseados em elos de corrente, buscou criar um fluxo contínuo ao acoplar rigidamente o fim da primeira etapa de uma tarefa ao início da segunda etapa de outra. Os testes mostraram que o acoplamento exato entre partes de diferentes tarefas, mostrou-se excessivamente restritivo. Mesmo as variações que permitiam pequenos espaços (gaps) ou utilizavam o aninhamento triplete não conseguiram superar a eficiência encontrada na V1, como fica evidenciado nos resultados na Figura 11.

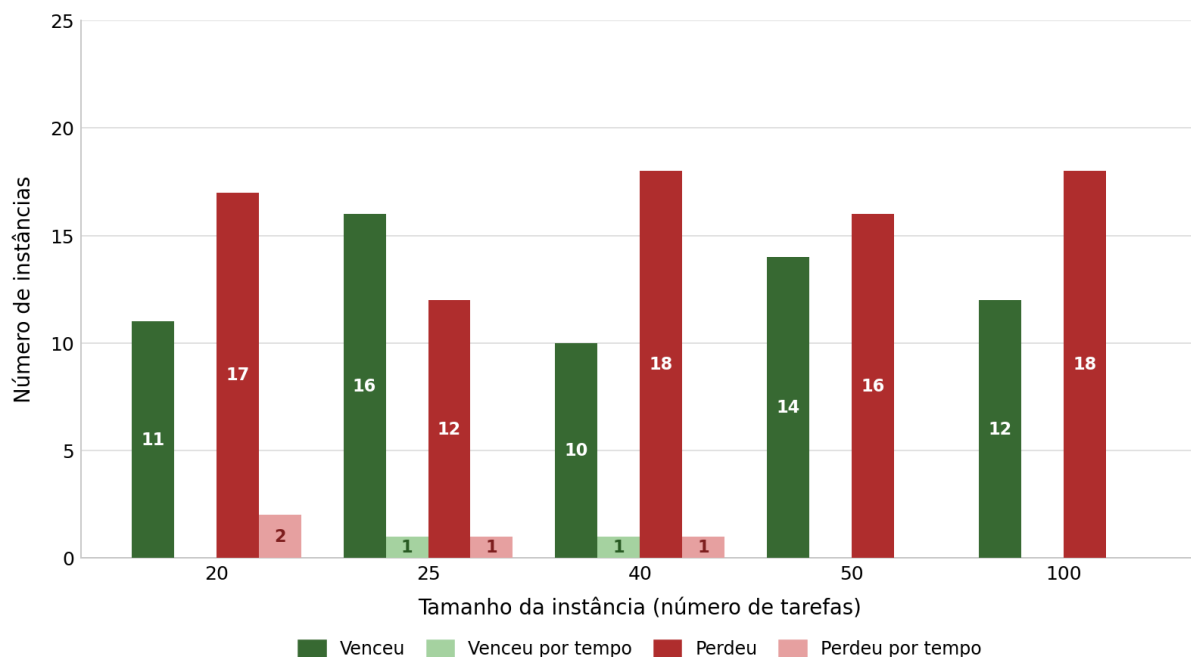


Figura 11: Resultados da execução da V2.3 (melhor variante dos modelos V2)

4.3. Desempenho da Heurística V1.4

Com essa abordagem, adotou-se a mesma lógica de divisão em etapas descrita na metodologia: o tempo de execução foi dividido em $n + 1$ etapas, sendo n o tamanho do bloco de tarefas candidatas ao encaixe. Em cada uma das n primeiras etapas, a modelagem era executada com uma das tarefas do bloco temporariamente removida, sendo posteriormente recolocada antes de se passar à etapa seguinte. Na etapa final, a melhor solução encontrada até esse momento, dentre as etapas anteriores, era utilizada como solução inicial para a execução da modelagem V1. E essa versão foi a que apresentou os melhores resultados, conforme mostra a Figura 12.

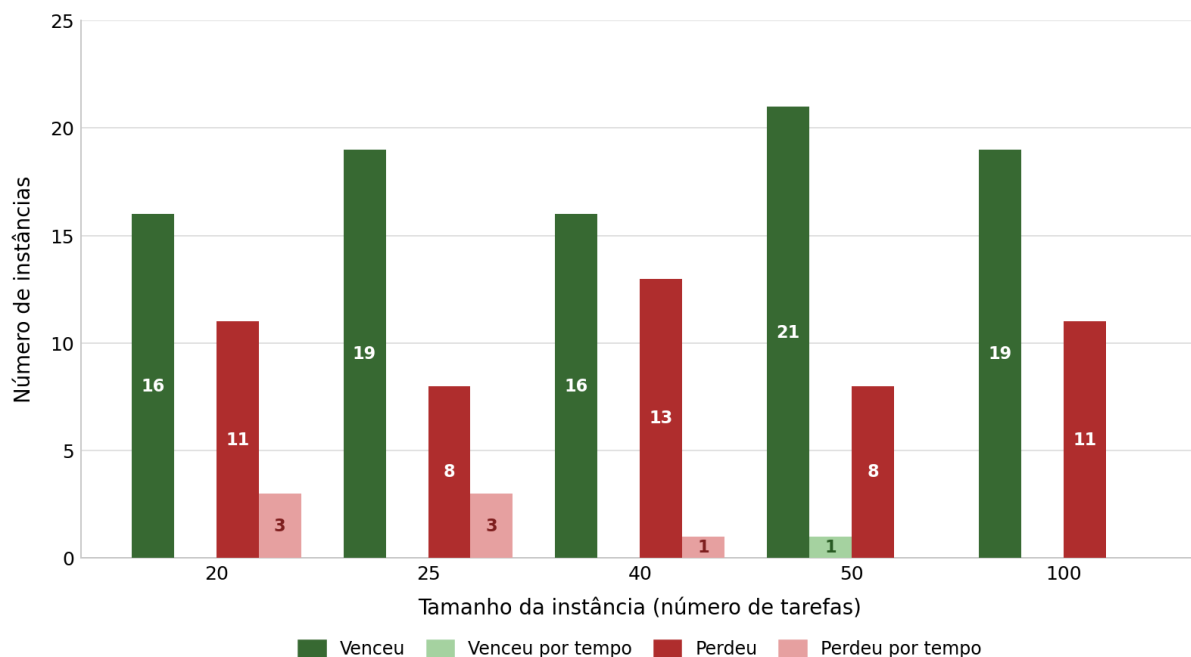


Figura 12: Resultados da execução da V1.4.

A variante V1.4 foi a abordagem que apresentou o melhor desempenho entre todas as heurísticas testadas, vencendo a modelagem base de Silva (2025) em 92 das 150 instâncias. Esse resultado indica que a estratégia de remoção e recolocação sistemática de tarefas, combinada ao uso da melhor solução parcial como ponto de partida para o solver, contribuiu de forma consistente para a qualidade das soluções encontradas, independentemente do tamanho da instância.

5 CONCLUSÃO

O presente trabalho investigou estratégias de otimização para o Coupled-Task Scheduling Problem (CTSP), um problema de escalonamento de alta complexidade e classificado como NP-Difícil. O objetivo central foi o desenvolvimento e o refinamento de heurísticas de encaixe voltadas para a minimização do makespan, aproveitando os intervalos fixos e obrigatórios entre as partes para maximizar a eficiência da máquina única.

Os resultados obtidos com a variante V1.4 mostraram que a remoção estratégica de tarefas do bloco de encaixe melhora o desempenho do modelo, enquanto restringir a busca a apenas uma fração das tarefas disponíveis — como considerar somente as 20% com os maiores intervalos — prejudica significativamente os resultados, sobretudo em instâncias de maior porte. Já os testes com a variante V2 mostraram que o posicionamento rígido entre

tarefas distintas frequentemente torna a execução inviável ou aumenta o makespan, devido ao conflito entre essas posições fixas e os tempos já ocupados por outras tarefas.

A principal contribuição deste trabalho foi o desenvolvimento da heurística V1.4. Ao testar a remoção e a recolocação de cada tarefa do bloco, uma por vez, e utilizar a melhor configuração encontrada como solução inicial para o *solver*, essa abordagem superou o desempenho das versões anteriores, tanto na V1 (e suas variantes), quanto na V2. Os resultados mostraram que essa estratégia foi capaz de superar a modelagem base de Silva (2025) na maioria das instâncias testadas, inclusive nas de maior porte, indicando que o uso de uma boa solução inicial contribui significativamente para a qualidade do resultado final.

Para trabalhos futuros, sugere-se investigar instâncias de maior porte ($N > 100$), bem como avaliar a viabilidade de utilizar múltiplos blocos-âncora simultaneamente em uma mesma instância — uma limitação deliberadamente adotada neste trabalho em favor da flexibilidade do modelo, mas que pode ser revisitada em busca de ganhos adicionais de desempenho.

Para fins de reprodutibilidade, o código-fonte das heurísticas desenvolvidas, bem como os scripts utilizados nos testes computacionais, está disponível publicamente em: https://github.com/PCanellas/tcc_si_2026.1.

REFERÊNCIAS

KHATAMI, Mostafa; SALEHIPOUR, Amir. A binary search algorithm for the general coupled task scheduling problem. **4OR**, v. 19, p. 593–611, 2021. Disponível em: <https://doi.org/10.1007/s10288-020-00463-w>.

KHATAMI, Mostafa; SALEHIPOUR, Amir; CHENG, T. C. E. Coupled task scheduling with exact delays: Literature review and models. **European Journal of Operational Research**, v. 282, n. 1, p. 19-39, 2020. Disponível em: <https://doi.org/10.1016/j.ejor.2019.08.045>.

SILVA, André Renato Villela da. Comparações de modelagens matemáticas para o Coupled-Task Scheduling Problem (CTSP). In: **57º SIMPÓSIO BRASILEIRO DE PESQUISA OPERACIONAL (SBPO)**, 2025, Gramado - RS. Anais... Gramado: SBPO, 2025.

SIMONIN, G.; DARTIES, B.; GIROUDEAU, R.; KÖNIG, J.-C. Isomorphic coupled-task scheduling problem with compatibility constraints on a single processor. **Journal of Scheduling**, v. 14, n. 5, p. 501-509, 2011..

SHAPIRO, R. D. Scheduling coupled tasks. *Naval Research Logistics Quarterly*, v. 27, n. 3, p. 477–481, 1980.

AGEEV, A.; KONONOV, A. Approximation algorithms for scheduling problems with exact delays. In: WORKSHOP ON APPROXIMATION AND ONLINE ALGORITHMS (WAOA), 2007, Eilat. *Lecture Notes in Computer Science*, v. 4927, p. 1–14. Berlin: Springer, 2008.

CONDOTTA, A.; SHAKHLEVICH, N. V. Scheduling patient appointments via multilevel template: A case study in chemotherapy. *Operations Research for Health Care*, v. 3, n. 3, p. 129–144, 2014.